

From Tuesday's Lab

1) logical expressions

Countdown

```
public static void countdown (int n) {  
    if (n == 0) {  
        print ("Blastoff!")  
        return;  
    }
```

```
}
```

```
    print(n);  
    countdown(n-1);
```

```
}
```

```
countdown(3);
```

What do you think this prints?

countdown(3)

print 3

countdown(2)

print 2

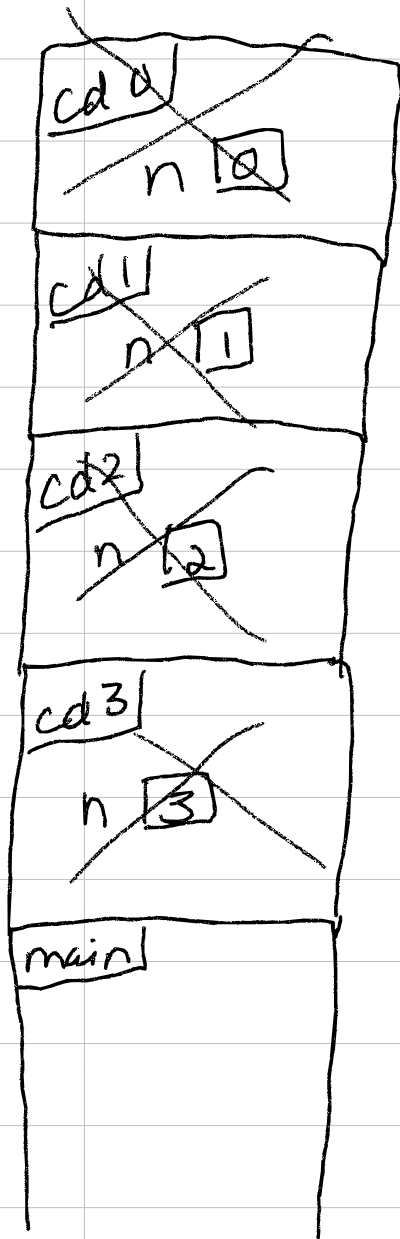
countdown(1)

print 1

countdown(0)

print blastoff!

* each call is a separate
method call



print blast off

print 1

print 2

print 3

Recipe for Recursion

1. Identify the simplest possible input.
2. write the answer for that case (base case)
3. figure out how to turn the problem into a smaller version of itself.
(recursive case)
4. Make sure the recursive call moves toward the base case.

* We solved this problem
by having the function
call itself.

* What if we want the
computer to repeat work
without making another
function call?

* remember: each recursive
call gets its own stack
frame

```
int n = 3 ;
```

```
while (n > 0) {
```

```
    System.out.println(n);
```

```
    n--;
```

```
}
```

- when code reaches the while loop, the condition is checked.

$n > 0?$ $3 > 0$ ✓

- then executes each line of the loop, then loops to the top of the while loop and checks the condition
- will loop until the condition is false.

```
public static void countdown(int n) {  
    while (n > 0) {  
        System.out.println(n);  
        n--;  decreases n  
                by 1  
    }  
    System.out.println("Blastoff");  
}
```

3 parts:

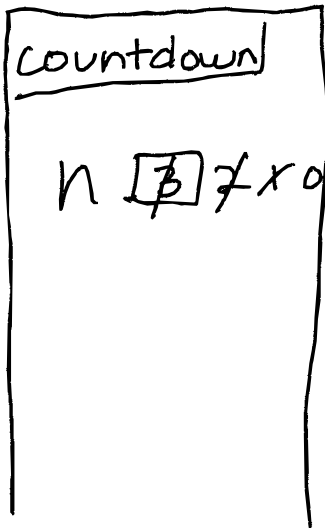
Set up: `int n = 3`

Condition: `while (n > 0)`

Update: `n--`

Let's draw the stack diagram...

say we call: `countdown(3);`



1st iteration: $n = 3$
check: $n > 0 \rightarrow 3 > 0 \checkmark$
print 3
 $n-- \rightarrow n = 2$

2nd iteration: $n = 2$
check: $n > 0 \rightarrow 2 > 0 \checkmark$
print 2
 $n-- \rightarrow n = 1$

3rd iteration: $n = 1$
check: $n > 0 \rightarrow 1 > 0 \checkmark$
print 1
 $n-- \rightarrow n = 0$

4th iteration: $n = 0$
check: $n > 0 \rightarrow 0 > 0 \times$
exit loop

Aside:

* we just used n--

* what is the -- doing?

* Shorthand for:

$$n = n - 1;$$

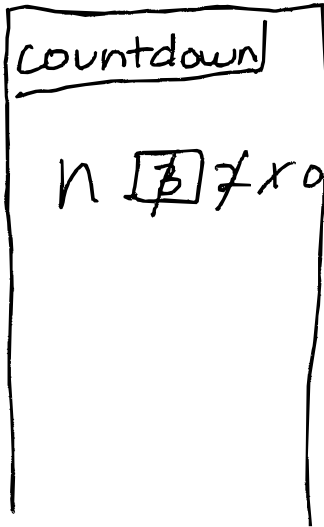
* we can also do n++

shorthand for:

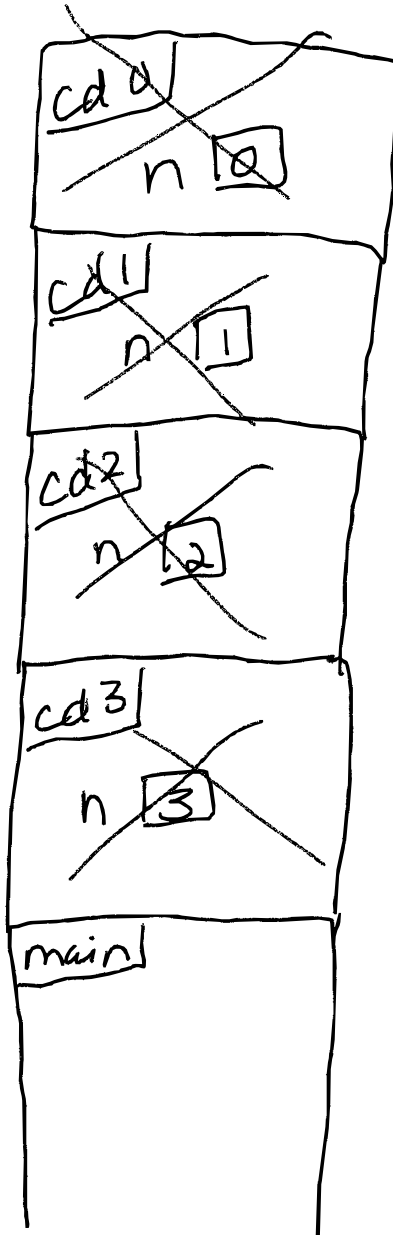
$$n = n + 1;$$

Compare Stack Diagrams

with
loop



print 3
print 2
print 1
blastoff!



print
blastoff!

print 1

print 2

print 3

Loops : more efficient,
everything stays in the
existing stack frame

Recursion : elegant, but
can take up a lot of
space on the stack

* many modern programming
languages compile recursive code
to iterative code

Print Stars

public static void printStars(int n)

→ Simplest input: 0

→ answer for base case:

just return

→ smaller problem:

print("*")

printStars(n-1)

```
public static void printStars(int n) {
```

```
    if (n == 0) {
```

```
        return;
```

```
    }
```

← not print line

```
    System.out.print("*");
```

```
    printStars(n-1);
```

```
}
```

print Stars(4) → *

print Stars(3) → **

print Stars(2) → ***

print Stars(1) → ****

Print stars

with while loop

```
int n = 3;
```

```
while (n > 0) {
```

```
    System.out.print("*");
```

```
    n--;
```

```
}
```

Returning a value

Let's make the return type int instead of void.

```
public static int sum(int n) {  
    if (n == 1) {  
        return 1;  
    }
```

```
    return n + sum(n-1);
```

```
}
```

* We are summing the #'s
from 1 to n.

$$\text{Sum}(4)$$

$$= 4 + \text{sum}(3)$$

$$= 4 + 3 + \text{sum}(2)$$

$$= 4 + 3 + 2 + \text{sum}(1)$$

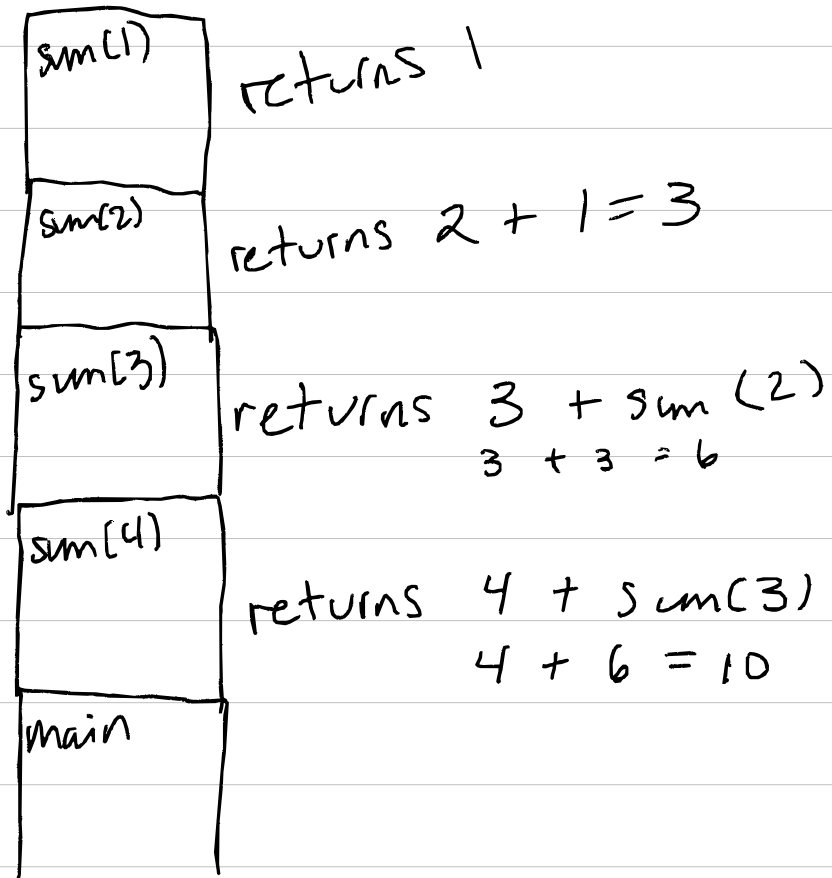
$$= 4 + 3 + 2 + 1$$

$$\text{sum}(1) = 1$$

$$\text{sum}(2) = 2 + 1 = 3$$

$$\text{sum}(3) = 3 + 3 = 6$$

$$\text{sum}(4) = 4 + 6 = 10$$



Sum $1 \dots n$

with while loop

```
int n = 3;  
int sum = 0;
```

```
while (n < 3) {
```

```
    sum += n;
```

```
    n++;
```

```
}
```

$$0 = 0 + 0$$

$$1 = 0 + 1$$

$$3 = 1 + 2$$

$$6 = 3 + 3$$

3
2
1

Factorial

base case : $n == 1$

recursive case : $n \times \text{fact}(n-1)$

```
public static int factorial(int n) {  
    if (n == 1) {  
        return 1;  
    }  
    return n * factorial(n-1);  
}
```

factorial(4)

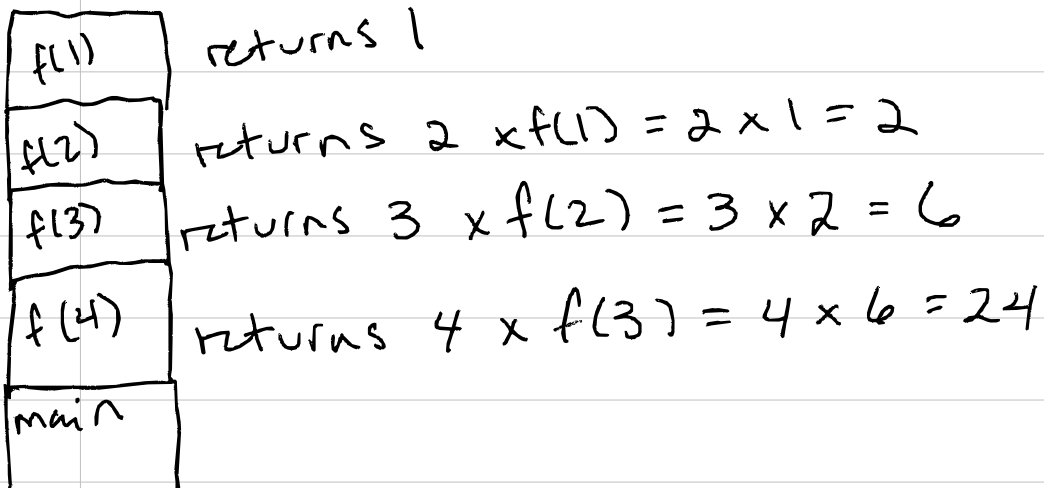
$4 \times \text{fact}(3)$

$4 \times 3 \times \text{fact}(2)$

$4 \times 3 \times 2 \times \text{fact}(1)$

$4 \times 3 \times 2 \times 1$

$= 24$



Factorial

with while loop

3 parts:

Set up: `int n = 3`

Condition: `while (n > 0)`

Update: `n--`

We keep doing this:

```
int n = 3;  
while (n > 0) {  
    // logic here  
    n--;  
}
```

Java gives us compact

Syntax for this:

for loop!

for(int i = 3 ; i > 0 ; i--)

↑
set up

↑
condition

↑
update